



AJEENKYA
DY PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

School of
Engineering

A Mini Project On
“Nutritional Breakdown of Foods”
For
Term Work Examination
**Bachelor of Computer Application in Artificial
Intelligence &
Machine Learning (BCA - AIML)**
Year 2025-2026
Ajeenkya DY Patil University
Submitted By-
Soniya Chauhan
Under the guidance
Of
Prof. Vivek More

Ajeenkya DY Patil University



D Y Patil Knowledge City,
Charholi Bk. Via Lohegaon,
Pune - 412105
Maharashtra (India)

Date: 14 / 04 / 2025

CERTIFICATE

This is to certified that, **Soniya Chauhan.**

A student of BCA(AIML) Sem-IV **URN No 2023-B-22042006A**

has successfully completed the dashboard report on

“Nutritional Breakdown of Foods”

As per the requirement of
Ajeenkya DY Patil University, Pune was carried out under my
supervision.

I hereby certify that; she has satisfactorily completed his Term-
Work Project work.

Place: Pune

Examiner

Sr No.	Index
1	Introduction & Objective
2	Methodology & Approach
3	Implementation & Code
4	Results & Visualizations
5	Conclusion & Future Scope

Introduction:

This dataset contains detailed nutritional information for a variety of food items. Each row represents a food product, and the columns provide nutritional facts such as:

- Food Name / Brand
- Serving Size
- Calories
- Macronutrients like:
 - Total Fat (and saturated/trans fat)
 - Cholesterol
 - Sodium
 - Total Carbohydrates (including dietary fiber and sugars)
 - Protein
- Micronutrients (e.g., Vitamin D, Calcium, Iron, Potassium)
- Possibly other attributes like food category, ingredients, or preparation style.

Purpose of the Dataset:

This kind of dataset is commonly used for-

- Building nutrition recommendation systems
- Meal planning apps
- Diet analysis and health monitoring
- Machine learning models to predict healthiness or suggest alternatives

- Labeling food products or checking for dietary compliance (e.g., low-sodium, keto-friendly)

Objective:

The objective of this dataset is to analyze the nutritional content of various food items to support dietary and health-related applications. With detailed information on vitamins, minerals, macronutrients, and caloric values, this dataset can be used to classify foods based on their nutritional profiles, recommend suitable food items for specific dietary needs, or predict caloric values using regression techniques. It also lends itself well to clustering similar food items to uncover patterns or groupings within food types. Additionally, it can assist in identifying foods with extreme nutrient levels, such as high fat or low protein, to support healthy eating recommendations. The presence of both raw and normalized food names allows for consistency checks or further analysis of text normalization effects. Overall, this dataset provides a strong foundation for both exploratory data analysis and predictive modeling in the field of nutrition and health.

Key objectives include:

1. Nutritional Classification-

Classify foods into categories (e.g., high/low protein, low-carb, low-fat) based on nutrient values.

2. Nutritional Recommendation System-

Build a system that recommends food items based on a user's dietary needs (e.g., high in Vitamin C, low in sodium).

3. Caloric Value Prediction-

Predict Caloric Value using other nutritional attributes using regression techniques.

4. Cluster Analysis-

Group similar food items together using unsupervised learning (e.g., k-means) to find food "types" based on nutrition.

5. Nutritional Profiling-

Analyze foods with extreme values (e.g., highest/lowest fat or sugar) to identify healthy/unhealthy options.

6. Compare Food Normalization-

If the food and food normalized columns differ, you could explore how normalization affects data consistency and modeling.

Methodology and Approach:

This project follows a structured data analysis workflow that includes data loading, cleaning, exploration, and visualization to understand the nutritional composition of various food items.

1. Data Loading and Preliminary Inspection

The dataset was imported using the pandas library. Initial exploration involved:

- Viewing the dataset's shape and structure.
- Checking for missing values.
- Displaying a sample of the data to get an understanding of its format.

This helped identify potential issues like duplicates, missing data, and non-numeric values in numeric fields.

2. Data Cleaning

To ensure accurate analysis, the dataset underwent several cleaning steps:

- Duplicate Removal: Duplicate rows were identified and dropped to avoid redundancy and bias in statistical summaries.

- Missing Value Handling:
 - Rows with more than 30% missing values were removed.
 - Remaining missing values were imputed with zeros, assuming negligible nutritional content.
- Data Type Conversion: Columns with object types were checked and converted to numeric types where applicable, to ensure compatibility with statistical functions and visualizations.

3. Descriptive Analysis

Basic statistical summaries (mean, std, min, max, etc.) were computed to understand the distribution and spread of nutritional variables.

4. Visualization and Nutritional Insights

A range of visualizations was created to explore relationships and highlight important features:

- High-Calorie Food Identification: Top 10 foods with the highest calorie values were listed and visualized using a histogram to show the overall calorie distribution.
- Protein Content Analysis: A bar chart highlighted the top 10 food items rich in protein, helping identify good protein sources.
- Sugar Content Distribution: A box plot was used to detect sugar content variability and outliers among food items.
- Correlation Analysis: A heatmap displayed the correlation between numeric nutrient variables. This helped uncover dependencies or patterns (e.g., high correlation between fat and calorie content).
- Caloric Value vs. Fat Analysis: A scatter plot showed the relationship between fat content and caloric value, allowing for visual assessment of their linear correlation.
- Univariate Distribution: Histograms with kernel density estimation (KDE) were generated for all numeric features to observe their individual distributions.

Conclusion of the Approach

This approach provides a comprehensive pipeline for cleaning, exploring, and analyzing nutritional data. It combines essential data preprocessing techniques with informative visualizations to uncover dietary patterns, identify key nutrients, and facilitate future tasks such as nutritional modeling or dietary recommendation systems.

Implementation & Code:

Step 1: Importing Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

- pandas: Essential for reading, manipulating, and analyzing structured data.
- matplotlib.pyplot: A basic plotting library used to create static, animated, and interactive visualizations.
- seaborn: Built on top of matplotlib, it simplifies creating attractive and informative statistical graphics.

Purpose: These libraries are foundational for data loading, cleaning, and visualization in Python.

Step 2: Load the Dataset

```
# Load the dataset
file_path = '/content/cleaned_nutrition_dataset[1].csv'
df = pd.read_csv(file_path)
```

- Loads the CSV file into a DataFrame named df.

Purpose: Brings the dataset into memory so we can perform operations on it.

Step 3: Display Basic Information

- `df.shape`: Returns number of rows and columns.
- `df.columns`: Lists all column names.
- `df.isnull().sum()`: Shows missing values in each column.
- `df.head()`: Displays the first 5 rows of the dataset.

Purpose: Helps understand the structure, identify missing data, and preview the dataset before processing.

Code:

```
# Display basic info
print("Dataset Shape:", df.shape)
print("\nInitial Columns:\n", df.columns)
print("\nInitial Missing Values:\n", df.isnull().sum())
print("\nSample Data:\n", df.head())
```

Output:

```
Dataset Shape: (3454, 13)

Initial Columns:
Index(['Vitamin C', 'Vitamin B11', 'Sodium', 'Calcium', 'Carbohydrates',
      'food', 'Iron', 'Caloric Value', 'Sugars', 'Dietary Fiber', 'Fat',
      'Protein', 'food_normalized'],
      dtype='object')

Initial Missing Values:
Vitamin C      0
Vitamin B11    0
Sodium         0
Calcium        0
Carbohydrates  0
food           0
Iron           0
Caloric Value  0
Sugars         0
Dietary Fiber  0
Fat            0
Protein        0
food_normalized 0
dtype: int64

Sample Data:
Vitamin C  Vitamin B11  Sodium  Calcium  Carbohydrates  \
0      0.082      0.086   0.018      2.8          0.073
1      0.400      0.005   0.065     10.2          3.700
2      0.000      0.000   0.000      0.0          0.000
3      0.000      0.000   0.000      0.0          0.000
4      0.000      0.000   0.000      0.0          0.000

food      Iron  Caloric Value  Sugars  Dietary Fiber  Fat  \
0  margarine with yoghurt   0.027      88.0      0.0          0.0   9.8
1  sunflower seed butter   0.700      99.0      1.7          0.9   8.8
2      hazelnut oil   0.000     120.0      0.0          0.0  13.6
3  menhaden fish oil   0.000    1966.0      0.0          0.0  218.0
4  cod liver fish oil   0.000     123.0      0.0          0.0  13.6

Protein      food_normalized
0   0.058  margarine with yoghurt
1   2.800  sunflower seed butter
2   0.000      hazelnut oil
3   0.000  menhaden fish oil
4   0.000    cod liver fish oil
```

Step 4: Data Cleaning Section

a. Remove Duplicates:

- Drops duplicate rows to ensure every record is unique.
- Reports how many rows were removed.

Purpose: Removes redundant data that could skew the analysis.

Code:

```
# 1. Remove duplicates
initial_count = df.shape[0]
df.drop_duplicates(inplace=True)
print(f"\nRemoved {initial_count - df.shape[0]} duplicate rows.")
```

Output:

```
Removed 0 duplicate rows.
```

b. Handle Missing Values:

- `dropna(thresh=...)`: Keeps only rows with at least 70% non-null values.
- `fillna(0)`: Fills remaining missing values with 0.

Purpose: Ensures sufficient data per row and fills blanks under the assumption that 0 means "not present" or "negligible."

Code:

```
# 2. Handle missing values
missing_before = df.isnull().sum().sum()
df.dropna(thresh=int(0.7 * df.shape[1]), inplace=True) # Drop rows with <70% non-null values
df.fillna(0, inplace=True) # Fill remaining missing values with 0
missing_after = df.isnull().sum().sum()
print(f"Missing values before cleaning: {missing_before}, after cleaning: {missing_after}")
```

Output:

```
Missing values before cleaning: 0, after cleaning: 0
```

c. Convert Object Columns to Numeric:

```
# 3. Convert numeric columns where possible
for col in df.columns:
    if df[col].dtype == 'object':
        try:
            df[col] = pd.to_numeric(df[col])
        except:
            continue
```

- Attempts to convert columns with text (object type) into numeric format.
- Ignores columns that can't be converted (like names).

Purpose: Ensures all numerical data is in proper format for statistical analysis and plotting.

Step 5: Descriptive Statistics

- `df.describe()`: Shows mean, standard deviation, min, max, and quartile values for each numeric column.

Purpose: Summarizes the key characteristics of the data numerically.

Code:

```
# Summary stats
print("\nDescriptive Statistics:\n", df.describe())
```

Output:

```

Descriptive Statistics:
      Vitamin C  Vitamin B11      Sodium      Calcium  Carbohydrates \
count  3454.000000  3454.000000  3454.000000  3454.000000  3454.000000
mean     9.501021    11.447140   141.437806    53.953960    18.724570
std     70.387542    33.849817   973.168394   103.339195    26.232674
min      0.000000     0.000000     0.000000     0.000000     0.000000
25%      0.000000     0.029000     0.057000     4.647500     1.700000
50%      0.300000     0.075000     0.200000    21.940000     9.110000
75%      5.382500     4.327500    21.372500    60.600000    26.400000
max     3872.000000  550.520000  14174.590000  1283.500000   390.200000

      Iron  Caloric Value      Sugars  Dietary Fiber      Fat \
count  3454.000000  3454.000000  3454.000000  3454.000000  3454.000000
mean     1.627954    227.591546     5.693513     2.157695    12.006807
std     4.384574    336.277025    13.136356     4.748876    26.726515
min      0.000000     0.000000     0.000000     0.000000     0.000000
25%      0.200000     60.050000     0.000000     0.000000     0.500000
50%      0.700000    134.415000     0.920000     0.710000     4.105000
75%      1.600000    285.007500     5.207500     2.327500    12.500000
max     121.200000   6077.000000    291.500000    76.500000   550.700000

      Protein
count  3454.000000
mean     10.819314
std     27.250089
min      0.000000
25%      1.262500
50%      3.800000
75%      9.242500
max     560.300000

```

Step 6: Data Analysis

a. Top 10 High-Calorie Foods

```
# Top 10 high-calorie items
if 'Calories' in df.columns:
    print("\nTop 10 High Calorie Foods:")
    print(df[['Calories']].sort_values(by='Calories', ascending=False).head(10))
```

- Sorts and displays the top 10 foods with the highest calorie values.

Purpose: Helps identify calorie-dense foods.

b. Calorie Distribution Plot

```
# Plot calorie distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['Calories'], bins=30, kde=True, color='orange')
plt.title('Calories Distribution')
plt.xlabel('Calories')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.show()
```

- Creates a histogram with 30 bins and a KDE (smoothed curve).

Purpose: Visualizes how calories are distributed across all foods.

Results and Visualization

a.Top 10 Foods by Protein Content

Displays a horizontal bar chart of the top 10 high-protein foods.

Purpose: Highlights the best protein sources in the dataset.

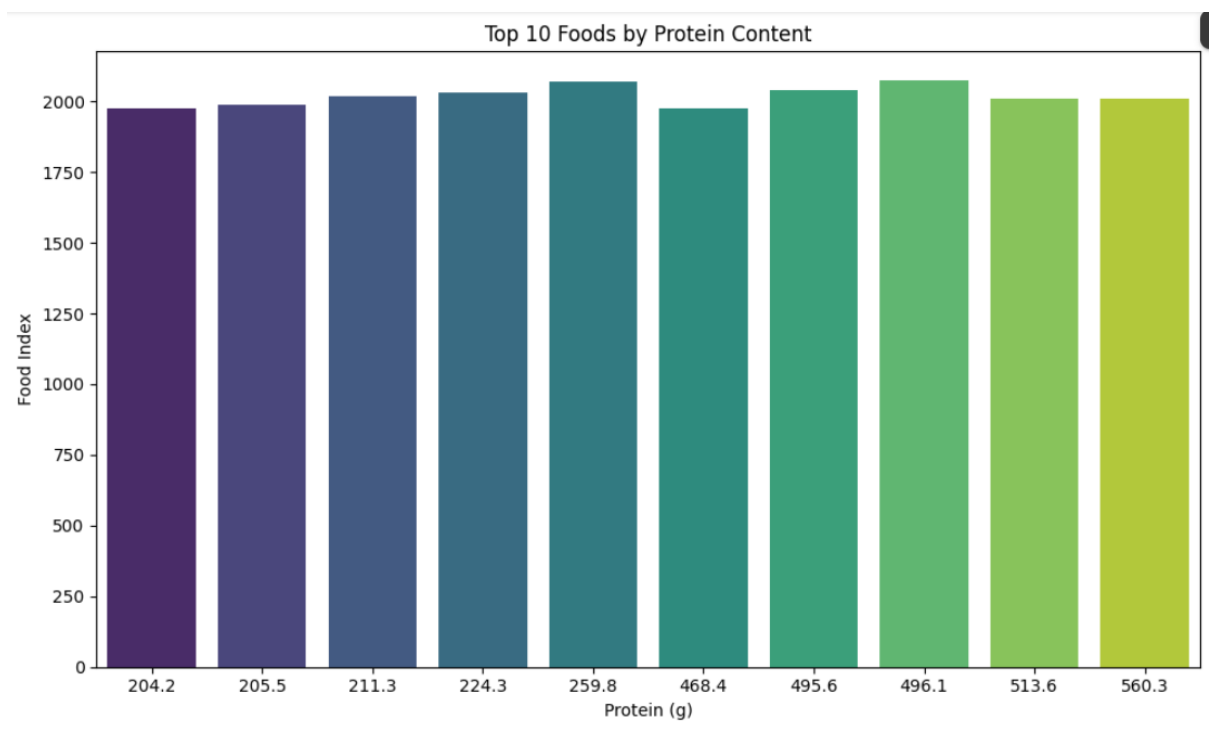
Code:

```

# Bar Plot: Top 10 foods by Protein content
if 'Protein' in df.columns:
    top_protein = df[['Protein']].sort_values(by='Protein', ascending=False).head(10)
    plt.figure(figsize=(10, 6))
    sns.barplot(x=top_protein['Protein'], y=top_protein.index, palette='viridis')
    plt.title('Top 10 Foods by Protein Content')
    plt.xlabel('Protein (g)')
    plt.ylabel('Food Index')
    plt.tight_layout()
    plt.show()

```

Output:



b.Box Plot of Sugar Content

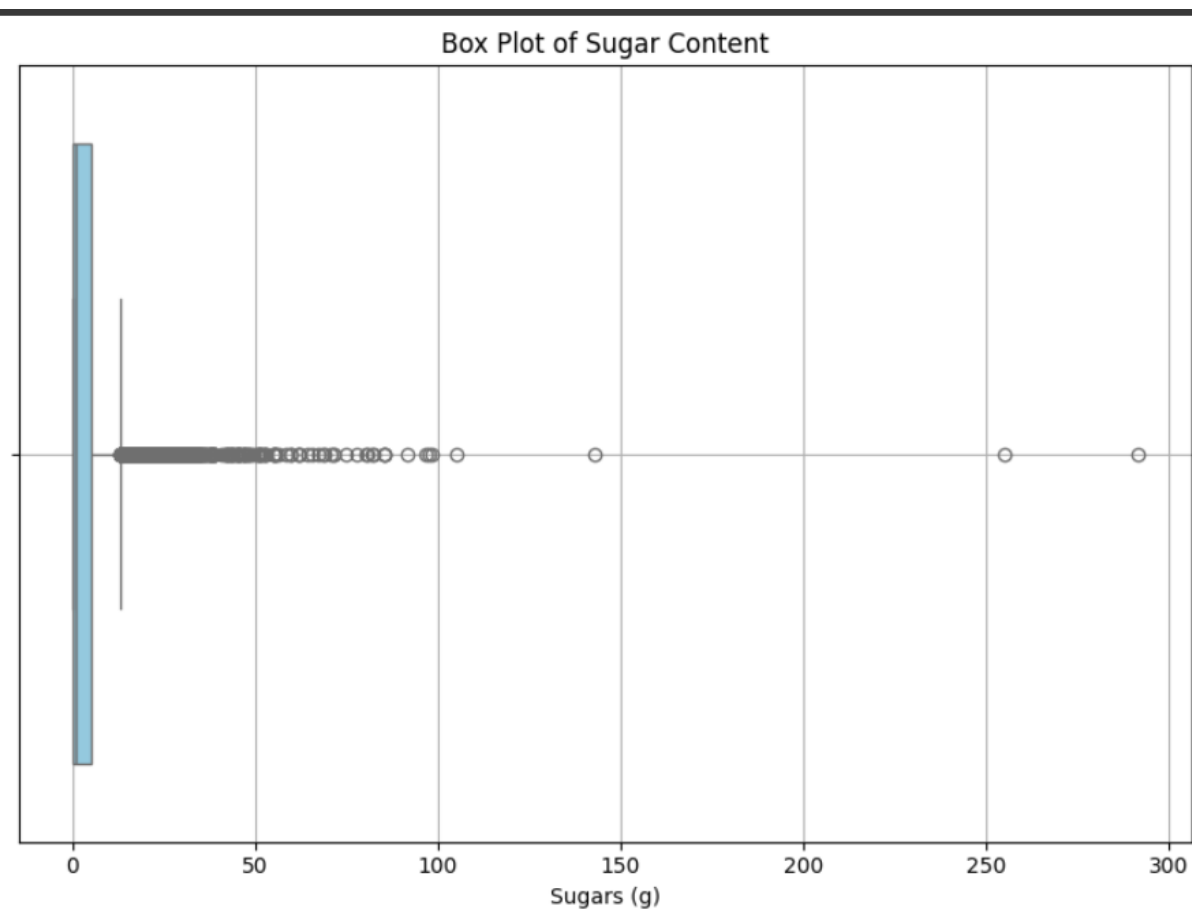
Plots a boxplot showing the distribution of sugar content.

Purpose: Detects variability, central tendency, and outliers in sugar levels.

Code:

```
# Box Plot: Distribution of Sugar content
if 'Sugars' in df.columns:
    plt.figure(figsize=(8, 6))
    sns.boxplot(x=df['Sugars'], color='skyblue')
    plt.title('Box Plot of Sugar Content')
    plt.xlabel('Sugars (g)')
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

Output:



c. Correlation Heatmap

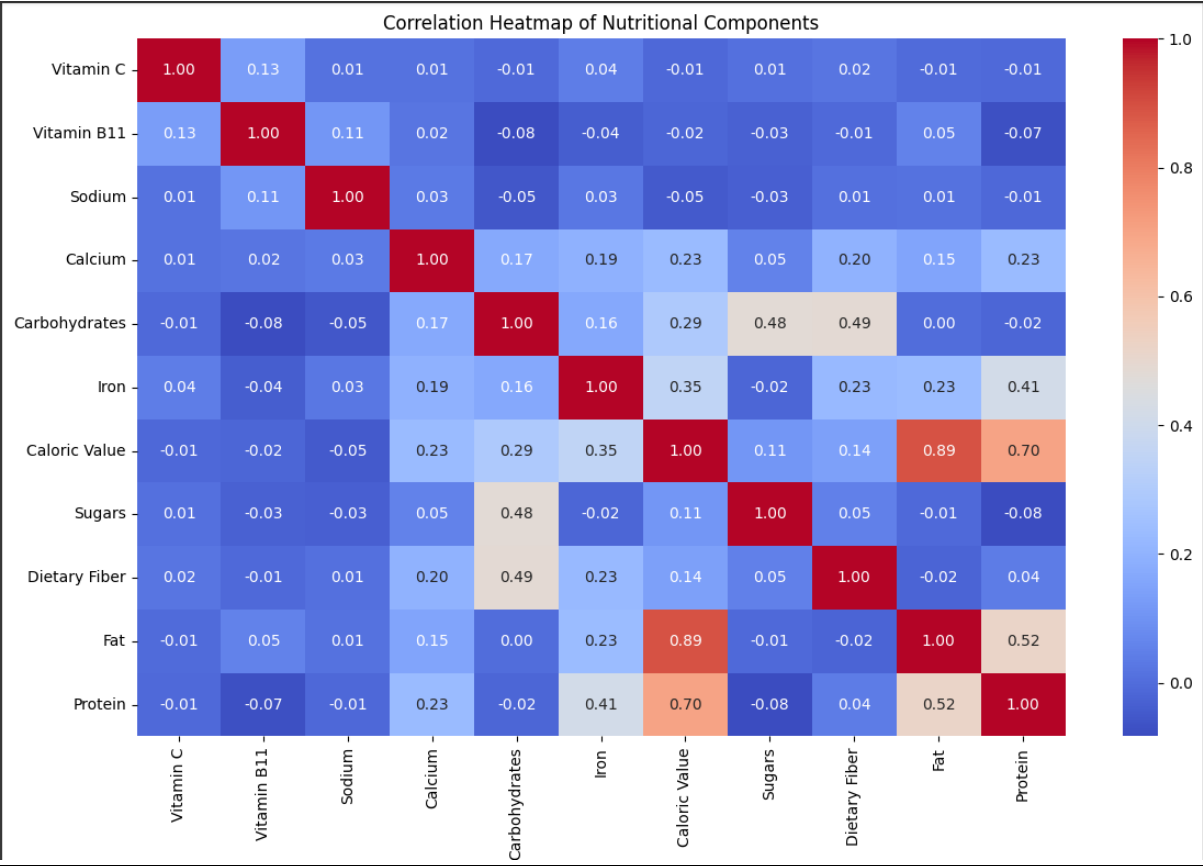
- Computes correlation between all numeric features and plots them as a heatmap.
- Darker/lighter colors indicate strength and direction of correlation.

Purpose: Finds relationships (e.g., strong correlation between fat and calories).

Code:

```
# Heatmap: Correlation between numeric nutrients
plt.figure(figsize=(12, 8))
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
correlation = df[numeric_cols].corr()
sns.heatmap(correlation, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation Heatmap of Nutritional Components')
plt.tight_layout()
plt.show()
```

Output:



d. Scatter Plot: Calories vs Fat

- Plots fat content against caloric value.
- Each dot represents a food item.

Purpose: Visualizes how fat contributes to total calories.

Code:

```
# Scatter plot: Calories vs Fat
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='Fat', y='Caloric Value', hue='food_normalized', legend=False)
plt.title('Calories vs Fat Content')
plt.xlabel('Fat (g)')
plt.ylabel('Caloric Value')
plt.show()
```

Output:



e. Histograms for All Numeric Columns

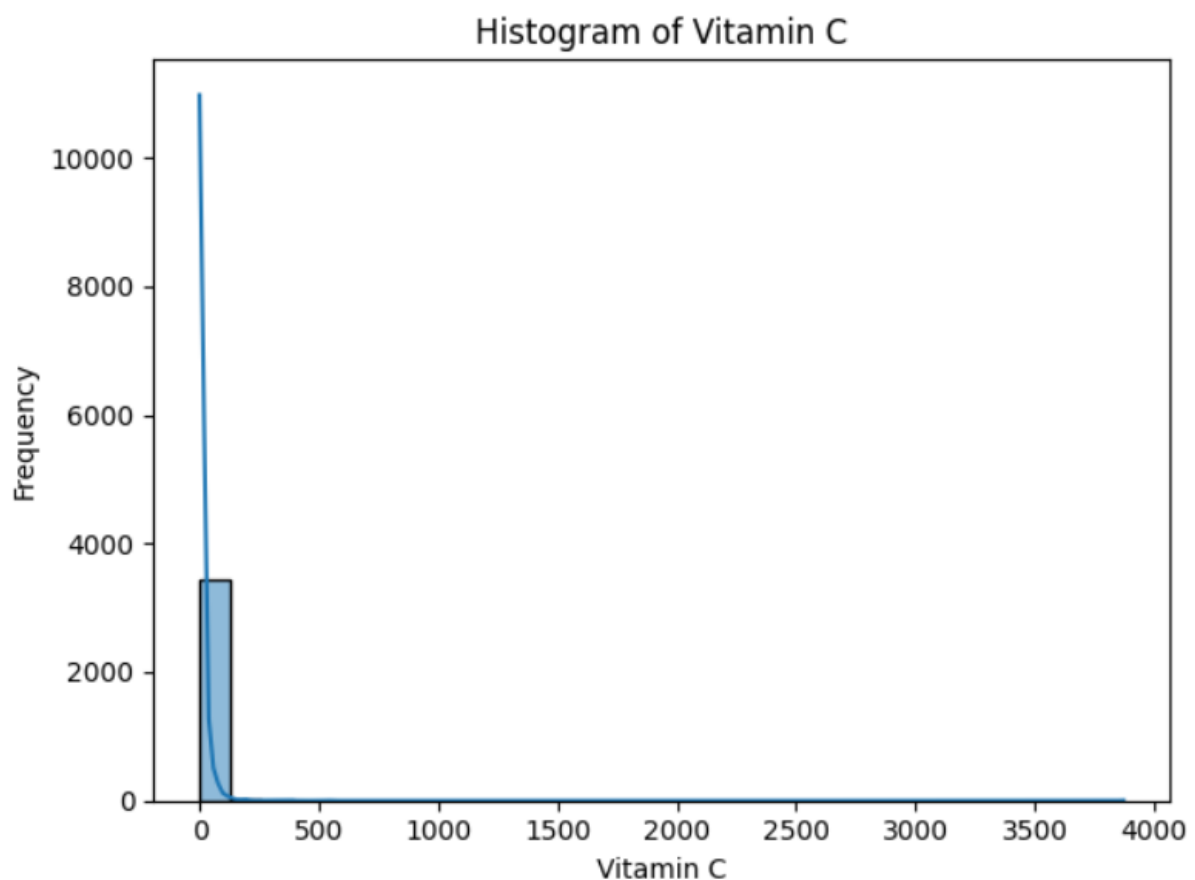
Iteratively plots histograms for every numeric column.

Purpose: Helps understand the distribution of each nutrient individually (normal, skewed, etc.).

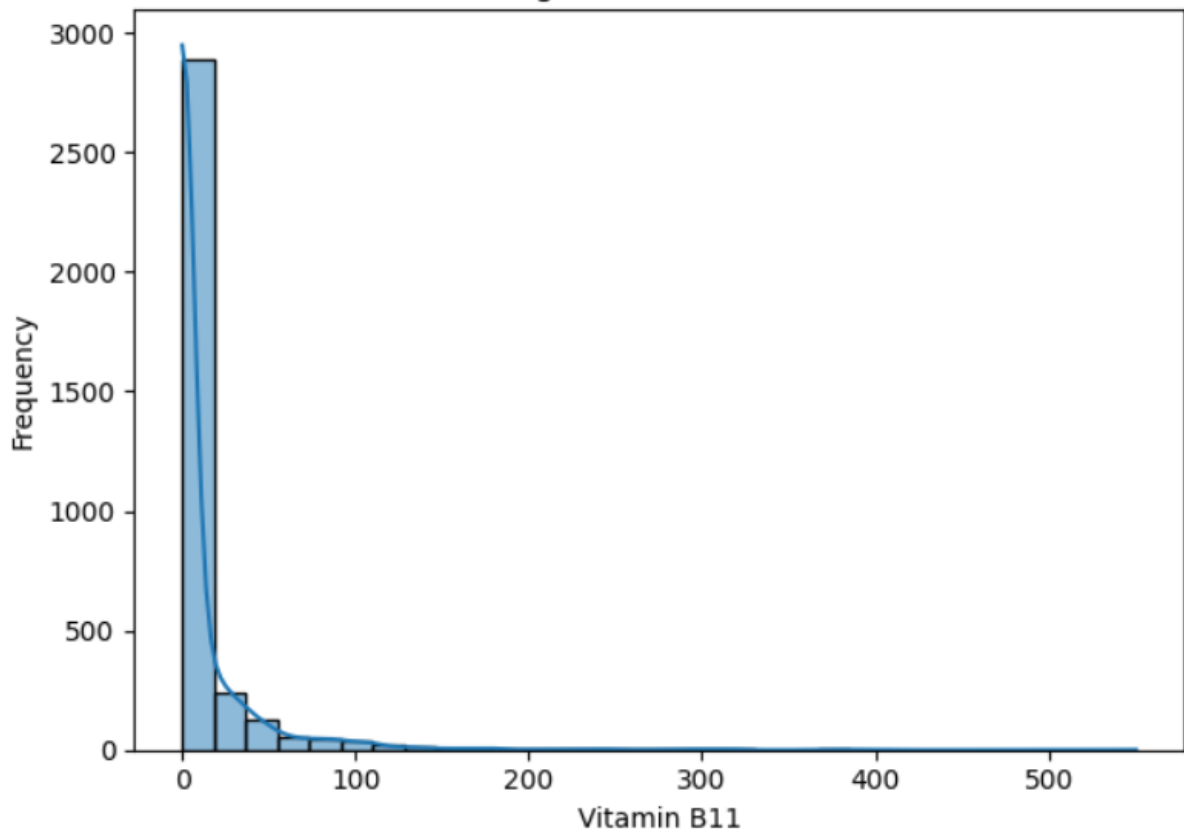
Code:

```
# Histograms for all numeric features
for col in numeric_cols:
    plt.figure()
    sns.histplot(df[col], bins=30, kde=True)
    plt.title(f'Histogram of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
```

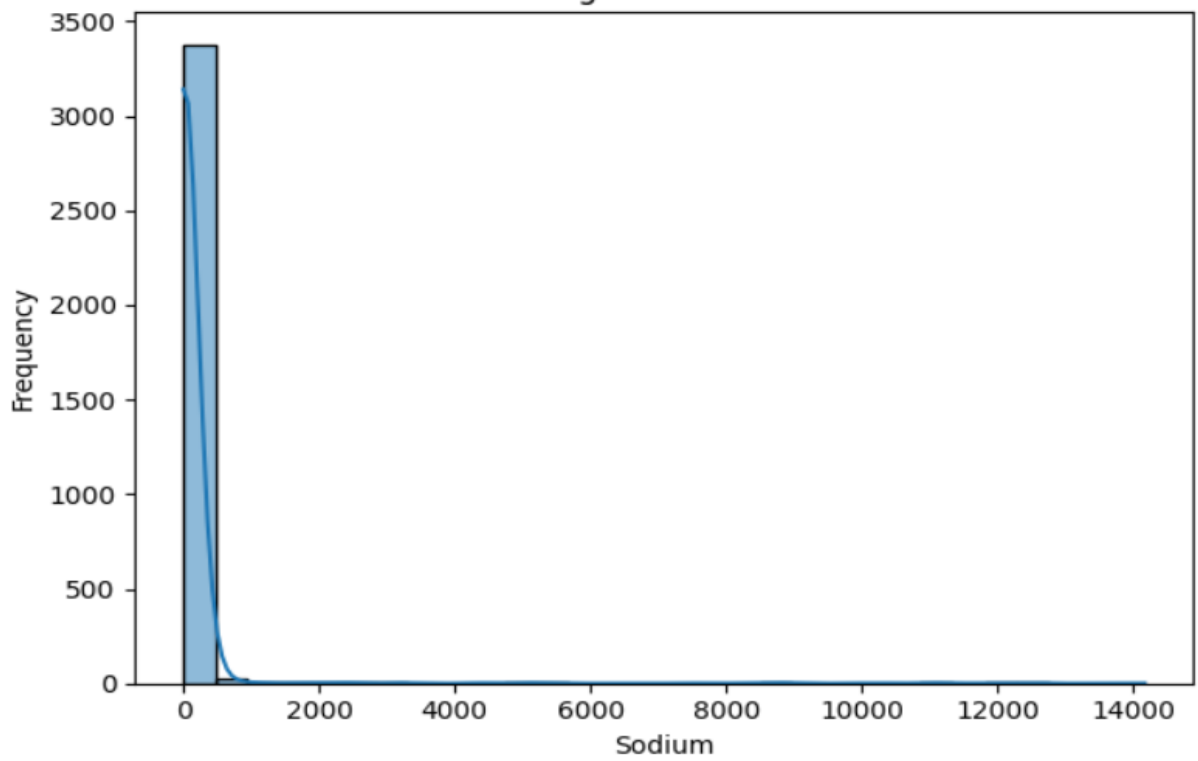
Output:



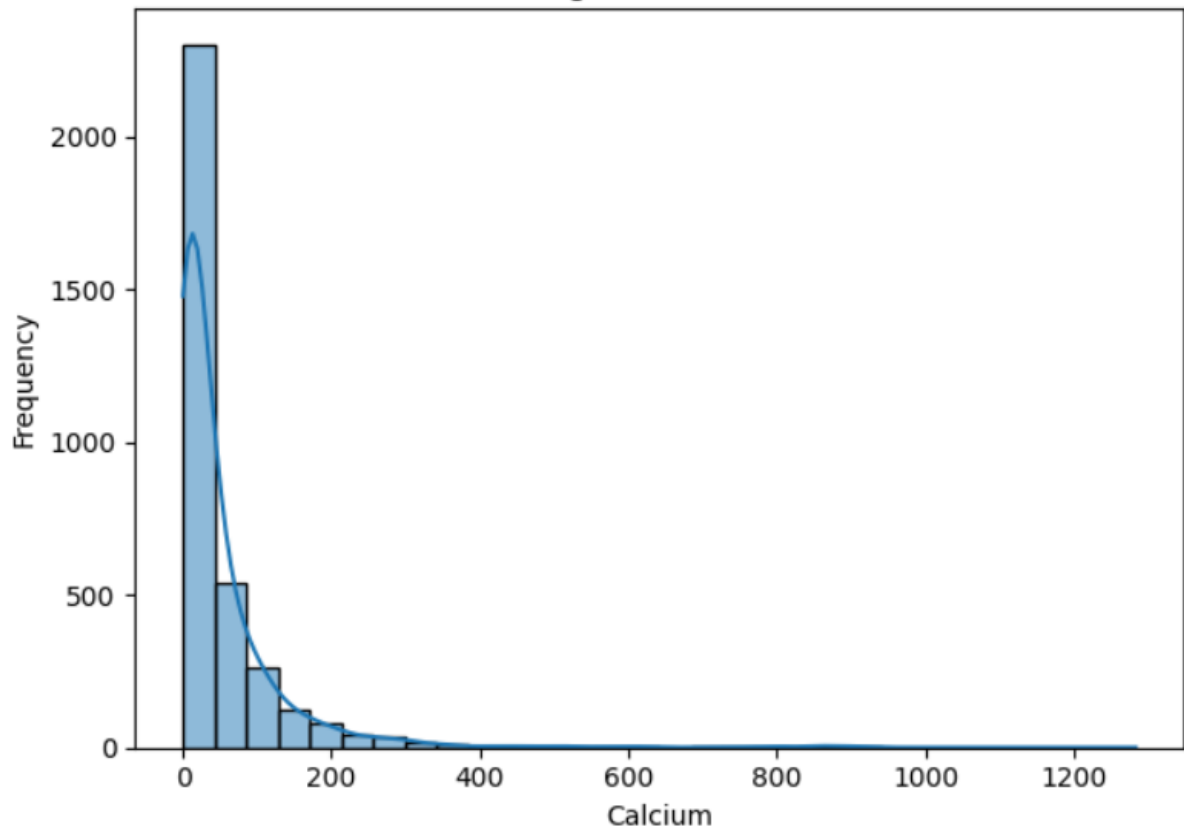
Histogram of Vitamin B11



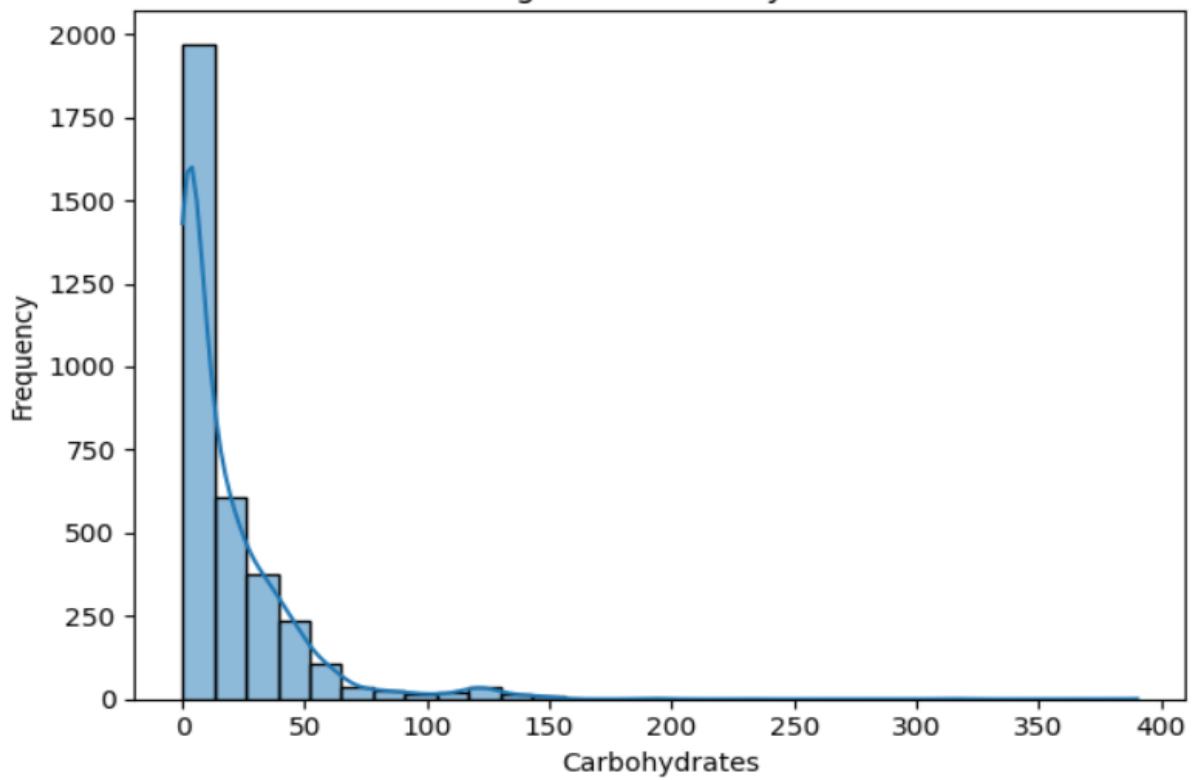
Histogram of Sodium

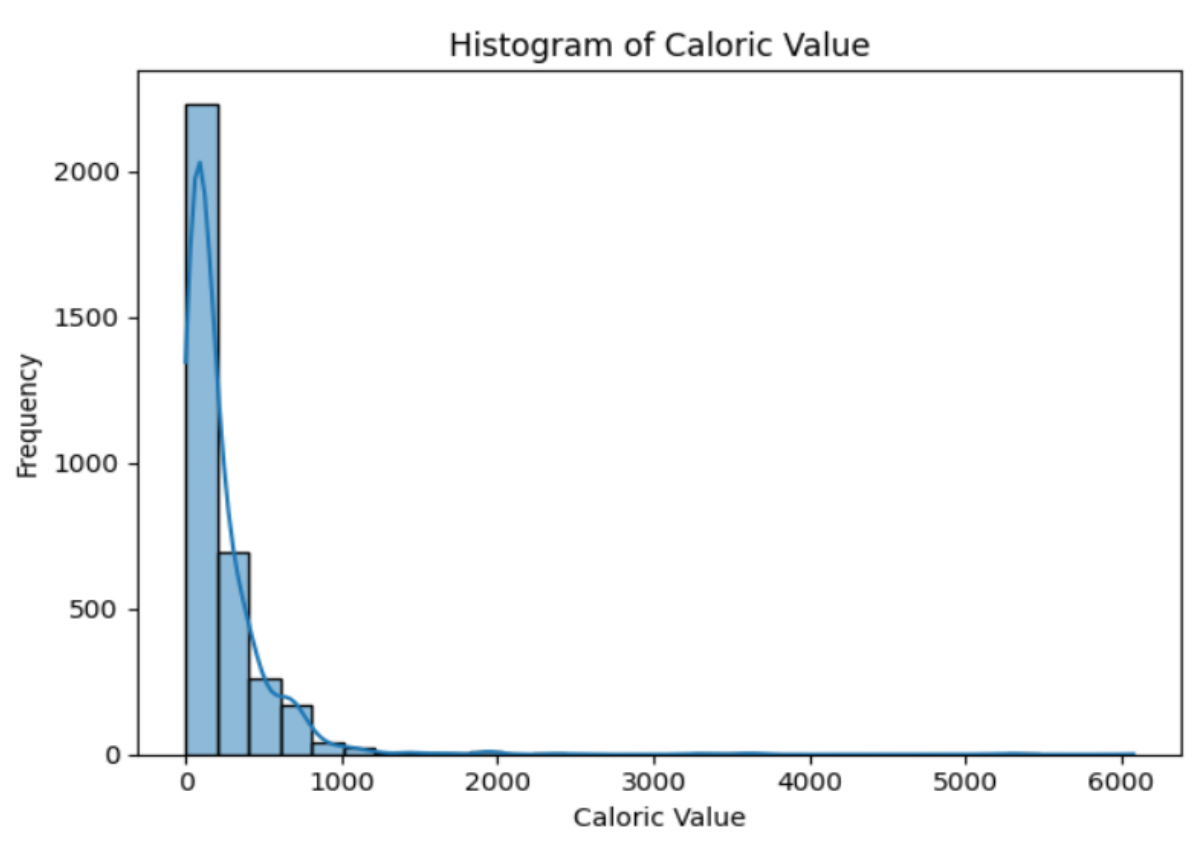
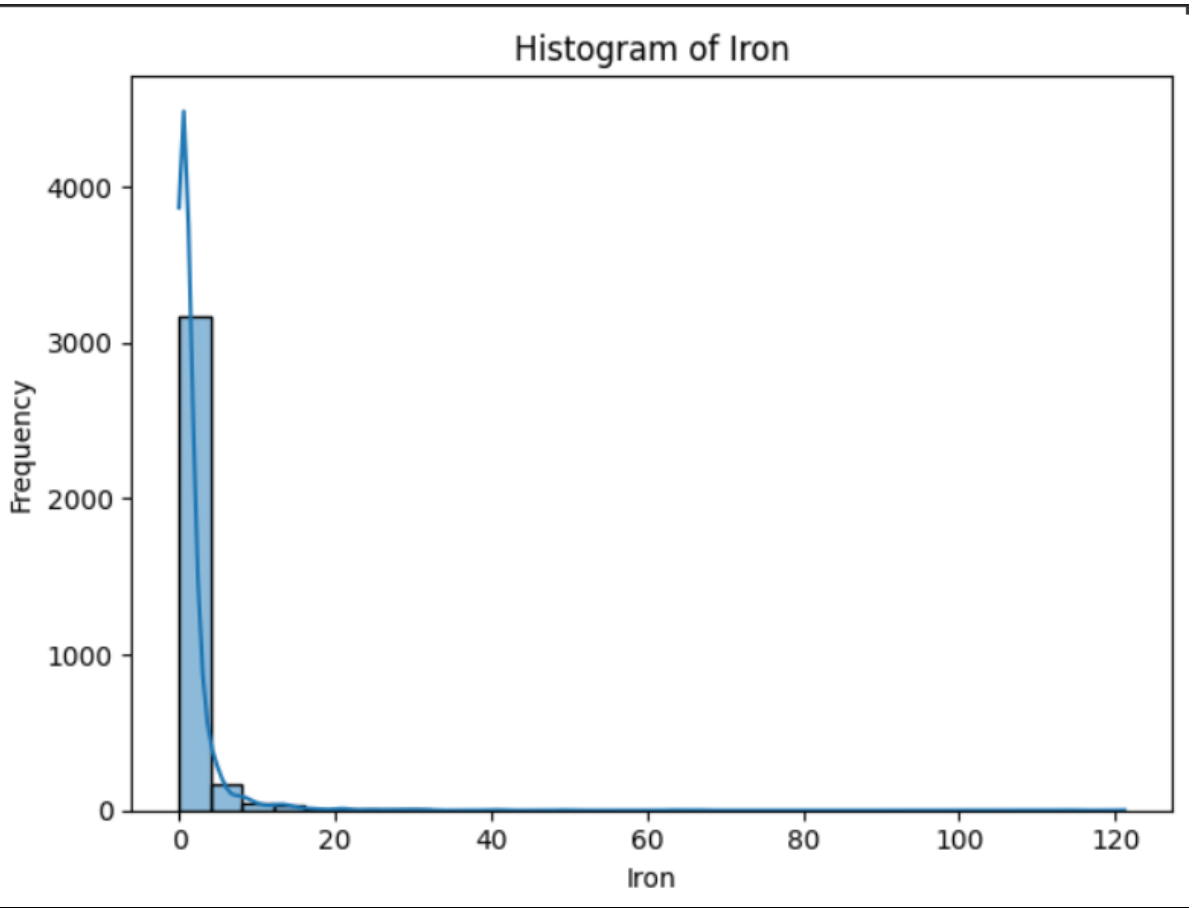


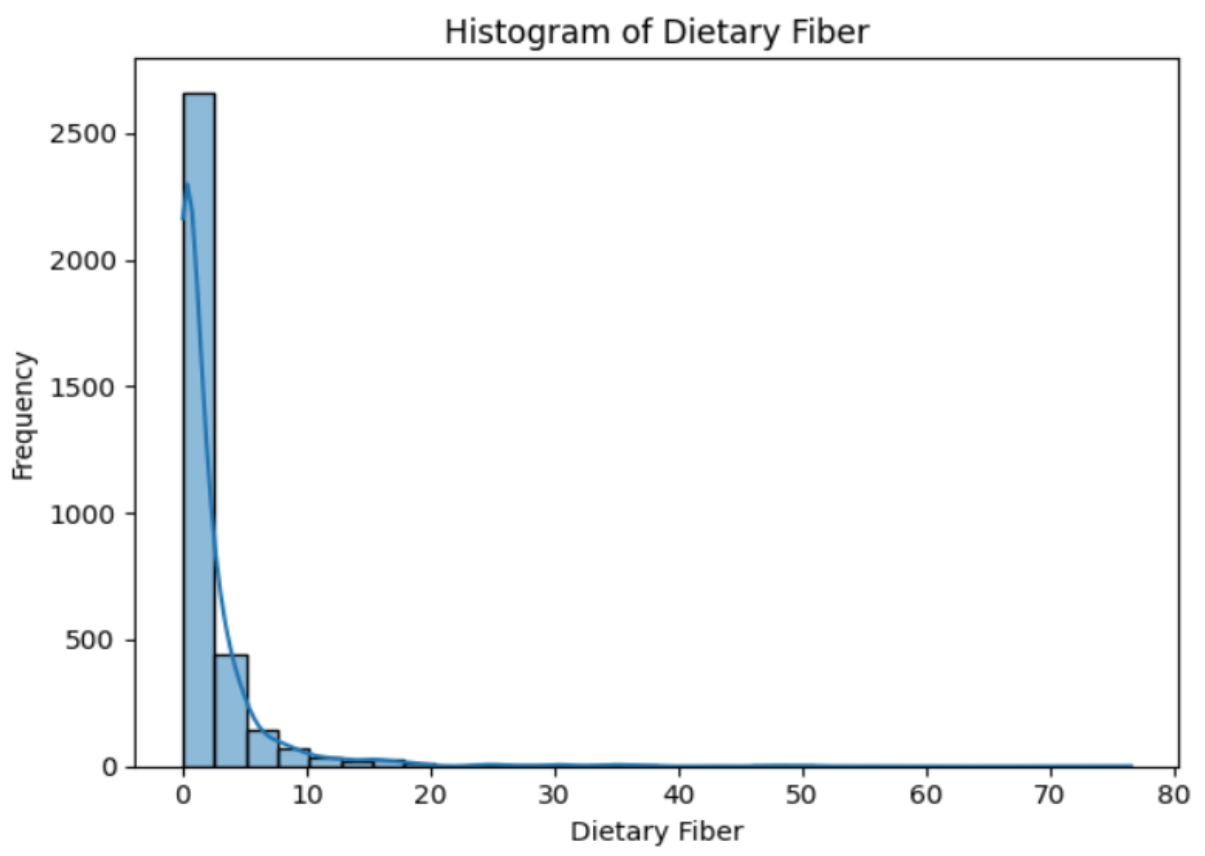
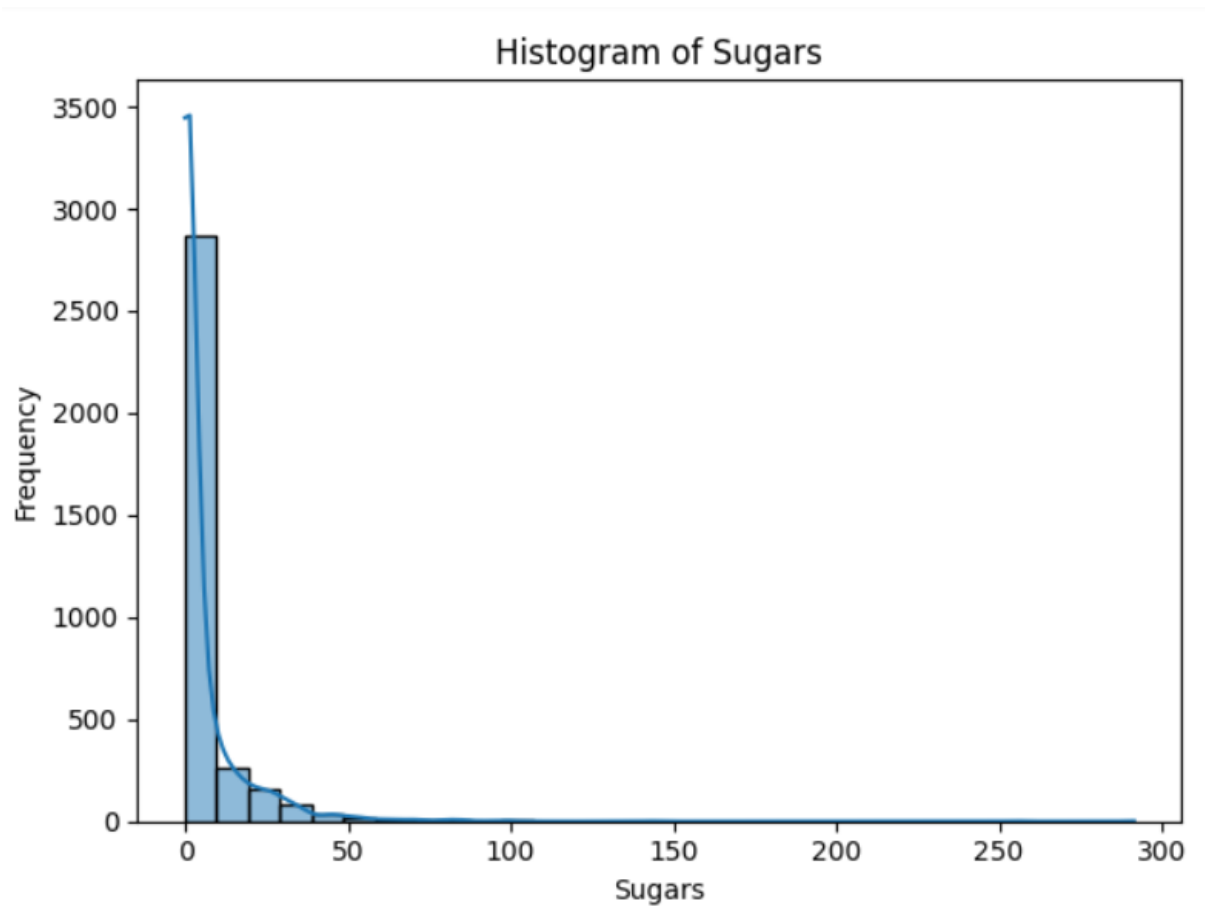
Histogram of Calcium



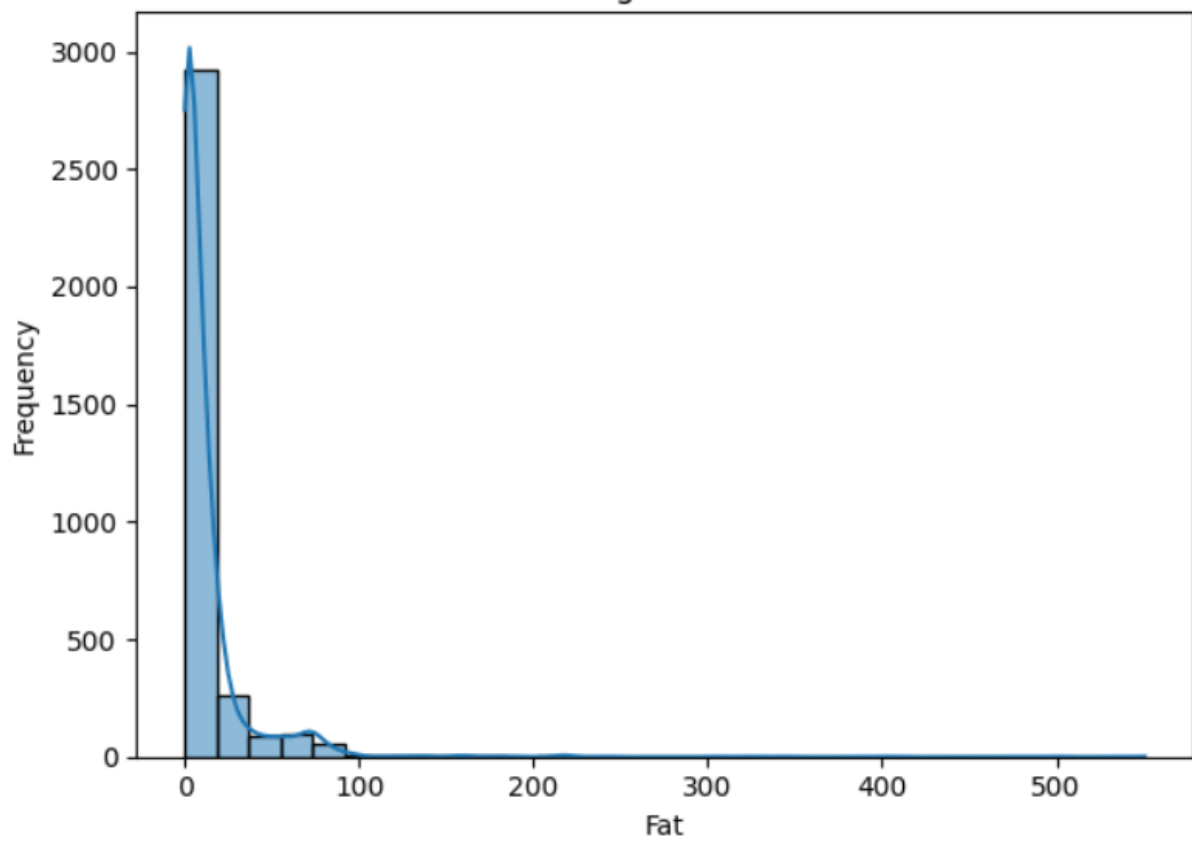
Histogram of Carbohydrates



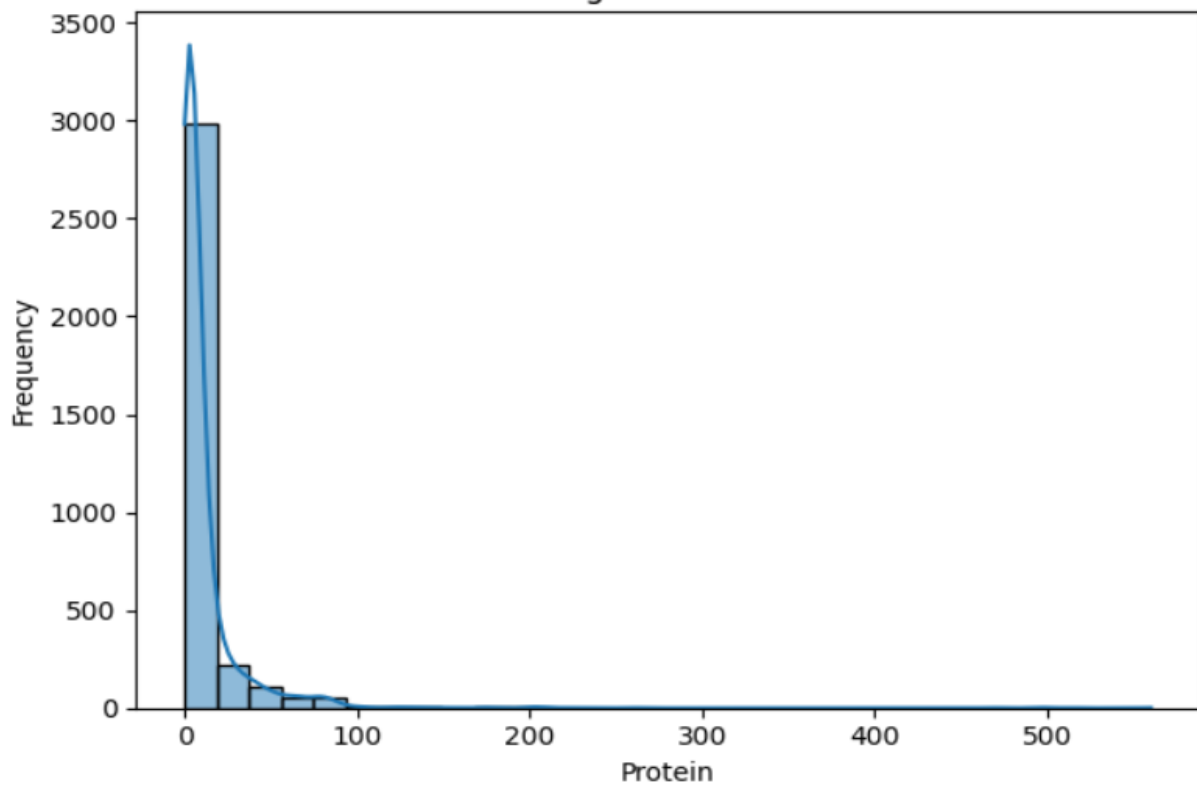




Histogram of Fat



Histogram of Protein



Future Scope & Conclusion:

Future Scope:

1. Dietary Recommendation Systems

- Develop models that suggest food items based on personal health goals (e.g., weight loss, muscle gain, diabetes management).
- Use user inputs like age, activity level, dietary restrictions to personalize recommendations.

2. Food Classification

- Classify foods into categories (e.g., high-protein, low-fat, high-carb) using clustering or supervised learning.
- Could be used for smart labeling or health app integrations.

3. Predictive Modeling

- Predict missing nutrient values using regression or machine learning models.
- Forecast nutritional impact over time if dietary habits are tracked.

4. Integration with Health APIs

- Combine this dataset with real-world health or fitness trackers for more actionable insights.

5. Deep Learning for Image Recognition

- Extend the dataset with food images and train models to recognize foods and estimate nutritional values directly from photos.

6. Time-Based or Regional Analysis

- If time-stamped or geotagged data is available, analyze how food consumption and nutrition trends vary by region or season.

Conclusion:

This analysis of the cleaned nutrition dataset has provided valuable insights into the composition and distribution of various nutrients across different food items. Through systematic data cleaning and exploratory data analysis (EDA), we were able to identify:

- The most calorie-dense and protein-rich food items.
- The distribution patterns of key nutrients such as calories, protein, fat, sugars, and more.
- Potential outliers and inconsistencies in nutrient values (e.g., high sugar or fat outliers).
- Strong correlations between nutritional components (e.g., fat and calories), which can help understand how certain nutrients influence overall caloric value.

The visualizations such as histograms, scatter plots, heatmaps, and box plots enriched the understanding of the dataset, making it easier to interpret trends and nutritional relationships.